

Hotspot-Induced GPU Imbalance in Irregular Probabilistic Workloads

Manuel I. Capel¹[0000–0003–2449–4394] and Alberto Salguero-Hidalgo²[0000–0001–9221–7351]

¹ Department of Computer Languages and Systems, University of Granada, Spain
`manuelcapel@ugr.es`

² ETSII, Department of Computer Science and Programming Languages,
Universidad de Málaga, Spain `alberto.salguero@uma.es`

Abstract. Irregular probabilistic GPU workloads often generate evolving spatial hotspots that progressively reduce the utilisation of streaming multiprocessors (SMs) under static workload mappings. In probabilistic cellular automata (CA), stochastic activation gradually concentrates computation in specific regions, resulting in heterogeneous SM workloads, lower GPU execution efficiency, and increased execution imbalance.

Although memory-locality-oriented mappings maximise coalesced accesses, they become increasingly inefficient under concentrated activation patterns. Kernel-level profiling with **Nsight Systems** shows that the **transition** kernel dominates the performance difference between the evaluated mappings and that its execution time is strongly coupled to tumour growth. Experiments on an NVIDIA RTX GPU achieved speedups of up to 218× compared to a sequential baseline. The non-monotonic evolution observed in the *fine-grain regional mapping* (FRM) overhead provides new insights into the dynamic behaviour of irregular probabilistic GPU workloads, and suggests the need for adaptive, workload-specific mapping strategies in the future.

Keywords: GPU Computing · CUDA · Cellular Automata · Workload Imbalance · Occupancy · Hotspots · stencil workloads.

1 Introduction

GPU acceleration is widely used for large-scale stencil and spatial simulations [2, 3]. However, irregular probabilistic workloads often produce non-uniform activation patterns that change dynamically during execution.

In probabilistic cellular automata (CA), localised hotspots emerge and progressively concentrate computational activity in specific spatial regions. This increasing concentration results in heterogeneous workloads across GPU thread blocks, which in turn contribute to workload imbalance across streaming multiprocessors (SMs). As a consequence, static spatial mappings become less effective and can reduce achieved occupancy and overall kernel execution efficiency.

Conventional GPU mappings primarily optimise memory locality by using contiguous spatial partitions. While these mappings maximise coalesced memory access and cache efficiency, they often result in severe imbalances when hotspot

distributions evolve dynamically. Although fine-grained decompositions generally improve workload balance, they can reduce memory locality and introduce additional execution overheads.

This work investigates the relationship between probabilistic activation, hotspot concentration, and GPU execution behaviour. Rather than proposing a fully dynamic load-balancing system, this study aims to analyse how different static spatial mappings affect GPU utilisation and SM imbalance under irregular, probabilistic workloads. The study uses tumour-growth cellular automata as a representative irregular probabilistic stencil workload. The focus of this work is architectural rather than biomedical. The probabilistic formulation is based on the tumour growth cellular automata (CA) model introduced by Poleszczuk and Enderling [4].

2 Spatial Mapping Strategies

Contiguous Row Mapping (CRM) The first strategy involves partitioning the domain into contiguous row segments, which are then assigned to thread blocks. This preserves locality but concentrates hotspots into a subset of SMs.

Fine-Grained Regional Mapping (FRM) The second strategy involves partitioning the domain into smaller spatial regions, which are then distributed across thread blocks. This improves balance but reduces memory locality.

Hybrid Workload Dispersion (HWD) This strategy combines contiguous row locality and cyclic spatial dispersion. Contiguous row segments are distributed across multiple regions to reduce hotspot concentration while preserving locality. In the hybrid strategy, only the spatial assignment between regions and thread blocks is modified; the probabilistic update semantics remain unchanged.

Metric	Baseline HWD	
Achieved Occupancy (%)	48.83	97.80
Active Warps per SM	15.63	31.30
L1 Active Cycles ($\times 10^6$)	435.89	432.96
L2 Active Cycles ($\times 10^6$)	410.79	407.15
DRAM Active Cycles ($\times 10^6$)	621.93	631.61

Table 1. Comparison of occupancy and memory subsystem activity between the baseline contiguous mapping (CRM) and the proposed Hybrid Workload Dispersion (HWD)

We compared occupancy and memory-subsystem activity using kernel-level profiling to assess whether the proposed mapping affects the GPU memory hierarchy. As shown in Table 1, despite the significant increase in achieved occupancy, activity in the L1, L2 and DRAM subsystems remains largely unchanged.

The fact that activity in the memory hierarchy has barely changed indicates that the observed performance differences cannot be explained solely by changes in memory traffic. Rather, they suggest that the computational workload is distributed and executed differently on the GPU during the simulation.

3 Temporal Evolution of Kernel Performance

Kernel-level profiling reveals that the computational cost of the simulation increases alongside tumour growth. As the number of active cells increases, the ex-

ecution time of the transition kernel rises almost proportionally, yielding Pearson correlation coefficients of over 0.99 for both CRM and FRM. This strong correlation suggests that the progressive concentration of active cells is closely linked to the increasing computational cost throughout the simulation.

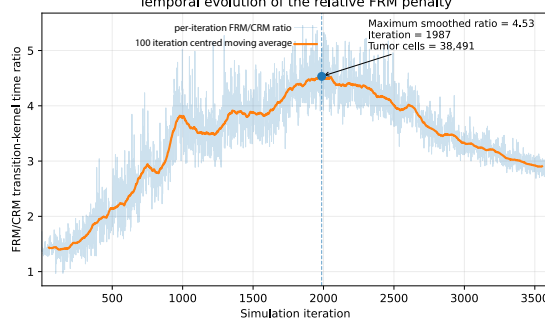


Fig. 1. Temporal evolution of the FRM/CRM transition-kernel execution-time ratio. The solid curve corresponds to the 100-iteration centred moving average

The **Transition** kernel accounts for approximately 81% of the total execution time for both mappings. The profiling results further show that execution time is strongly correlated with tumour growth (Pearson correlation coefficient > 0.99), while the relative FRM overhead exhibits a non-monotonic evolution, reaching its maximum at intermediate tumour sizes before decreasing during the late growth stages.

Figure 1 further analyses the relative behaviour of both mappings by showing the 100-iteration centred moving average of the FRM/CRM transition kernel execution time ratio. Unlike the absolute execution time, the relative FRM overhead does not increase monotonically. Instead, it reaches a maximum value of 4.53 at iteration 1987, when the simulated tumour contains 38,491 cells. Despite the continued increase in tumour size and kernel execution time, it then progressively decreases. This observation demonstrates that evaluating GPU mappings only at the end of the simulation or through aggregate execution times may conceal important temporal differences in their behaviour.

This change in trend suggests that the relative efficiency of the two mappings is influenced by factors beyond the overall workload size. The observed behaviour indicates an evolving computational structure during tumour growth that affects how each mapping exploits the GPU. While the underlying microarchitectural mechanisms cannot be determined from the current profiling data alone, this previously unreported temporal execution pattern highlights the potential value of adaptive workload-mapping strategies for irregular probabilistic GPU simulations.

4 Experimental Evaluation

Experiments were conducted using an NVIDIA RTX 3090 GPU with Ampere architecture (GA102), which has 82 SMs. The simulations used a 1024×1024 cellular lattice. CRM employs a one-dimensional CUDA grid of 1024 thread blocks

with 1024 threads per block; consequently, each lattice update is completed in a single pass. By contrast, FRM launches just 16 thread blocks with 1024 threads per block; each thread iterates over 64 spatial positions, necessitating 64 sequential iterations to update the entire lattice. *Nsight Systems* profiling was used to analyse kernel execution time and temporal execution behaviour.

The implementation achieved speedups of up to $218\times$ relative to the sequential baseline. Profiling revealed that kernel execution time increases almost proportionally with tumour growth, while the relative FRM overhead follows a non-monotonic evolution that peaks at intermediate tumour sizes. Hybrid dispersion achieved a more balanced workload distribution than the previous implementation [5]. The profiling data suggest that the increase in computational cost is associated with the progressive concentration of active cells and the resulting evolution of the workload.

5 Conclusions

Irregular probabilistic stencil workloads reveal a key trade-off between memory locality and balanced GPU execution. While purely locality-oriented mappings become increasingly inefficient as hotspots concentrate, fine-grained decompositions may sacrifice spatial locality to improve workload balance. This work analyses hotspot-induced imbalance in probabilistic GPU workloads. Hybrid workload dispersion provides a compromise between preserving spatial locality and improving workload balance across the GPU. An important outcome of this study is the identification of a previously unreported temporal execution pattern in GPU-based tumour growth simulations. Understanding why the relative overhead of FRM decreases during the late growth stages could provide valuable insights into designing adaptive load-balancing strategies that improve efficiency and scalability. This constitutes one of the main directions of our ongoing research. The source code and artefacts are publicly available [1].

References

1. Capel, M.I., Rodríguez Domingo, L.: Load balancing strategies for parallel tumour growth simulations. <https://github.com/mcapeltu/Load-Balancing-Strategies-for-Parallel-Tumor-Growth-Simulations> (2025), accessed: 2025-10-13
2. D’Ambrosio, D., Filippone, G., Rongo, R., Spataro, W., Trunfio, G.A.: Cellular automata and gpgpu: An application to lava flow modeling. *Int. J. Grid High Perform. Comput.* **4**, 30–47 (2012), <https://api.semanticscholar.org/CorpusID:7824277>
3. Gibson, M.J., Keedwell, E.C., Savić, D.A.: An investigation of the efficient implementation of cellular automata on multi-core cpu and gpu hardware. *J. Parallel Distributed Comput.* **77**, 11–25 (2015), <https://api.semanticscholar.org/CorpusID:22528249>
4. Poleszczuk, J., Enderling, H.: A high-performance cellular automaton model of tumor growth with dynamically growing domains (2013), <https://arxiv.org/abs/1309.6015>, arXiv preprint arXiv:1309.6015
5. Salguero, A.G., Capel, M.I., Tomeu, A.J.: Parallel cellular automaton tumor growth model. In: *Practical Applications of Computational Biology & Bioinformatics* (2018)